

# AMCAS Assignment 1

Memory Circuits and Systems: From Devices to Systems

Vyshnavi Tanikonda (2025900014)

## Setup & Configuration Details

All simulation source code, analysis scripts, raw data traces, and plotting configurations are fully reproducible and publicly available in the following GitHub repository: <https://github.com/VYSHNAVI2469/AMCAS--ASG1--Memory-sim>.

All simulations were conducted natively on an x86\_64 Linux environment (Ubuntu, WSL2). The tool configurations are as follows:

- **ngspice**: Predictive Technology Model (PTM) 45 nm bulk BSIM4 model card,  $V_{DD,nom} = 1.1$  V.
- **CACTI 7.0**: 45 nm ITRS High-Performance (**itrs-hp**) configuration at  $T = 350$  K, UCA model.
- **NVSim**: 1T-1MTJ STT-MRAM cell at 45 nm, current-sensing read, current-mode set/reset.
- **Ramulator**: DDR4-2400 8x8, single channel, FR-FCFS scheduler.
- **gem5 (v24.0.0.0)**: x86 ISA, Deriv03CPU (Out-of-Order) and TimingSimpleCPU (In-Order) with the GAPBS benchmark suite.

*Central architectural question carried through all five parts: **Should the 2 MB L2 cache in our accelerator use SRAM or STT-MRAM?***

## 1 Part A: 6T SRAM Bitcell Physics (ngspice)

The baseline 6T SRAM bitcell uses  $V_{DD} = 1.1$  V. The pull-down (MN1) to access-transistor (MA1) sizing ratio is  $\beta_r = W_{pd}/W_{ax} = 0.20/0.16 = 1.25$ . The cell is initialized with  $Q = 0$  V,  $QB = 1.1$  V, and both bitlines precharged to 1.1 V with  $C_{BL} = 180$  fF.

**Task 1 (Baseline read)**: At  $t = 2.0$  ns, the measured differential is  $\Delta V(BLB, BL) = 729.86$  mV, with the internal storage node peaking at  $v(q)_{max} = 205.52$  mV at  $t = 1.058$  ns. This is roughly  $10.6\times$  the  $\approx 69$  mV signal a 1T1C DRAM cell develops from passive charge sharing alone, since the SRAM cell *actively* discharges the bitline through MA1/MN2 rather than relying on charge redistribution.

**Task 2 (Widened access transistor)**: Growing the access transistors to  $W_{ax} = 0.24$   $\mu$ m drops the cell ratio to  $\beta_r = 0.20/0.24 = 0.833 < 1.0$ . The storage-node bump grows to  $v(q)_{max} \approx 272.0$  mV, consuming more than 60% of the inverter trip voltage ( $V_{trip} \approx 450$  mV). Under the ideal, noise-free transient conditions simulated, the disturbance still relaxes back toward 0 V rather than latching into a destructive flip – but with realistic threshold-mismatch ( $\sigma_{V_{th}} \approx 35$  mV from RDF), cells in the distribution tail would very plausibly trip MN1 and undergo a destructive read upset. (See Figure 1.)

**Task 3 ( $V_{DD}$  sweep,  $C_{BL} = 1$  pF)**: Sweeping  $V_{DD}$  from 1.1 V down to 0.6 V,  $\Delta V(BLB, BL)$  at  $t = 2.0$  ns falls monotonically from 143.59 mV to 26.91 mV. Using Lecture 5's 25 mV sense-amplifier offset and interpolating between the 0.65 V and 0.60 V points gives a failure boundary of

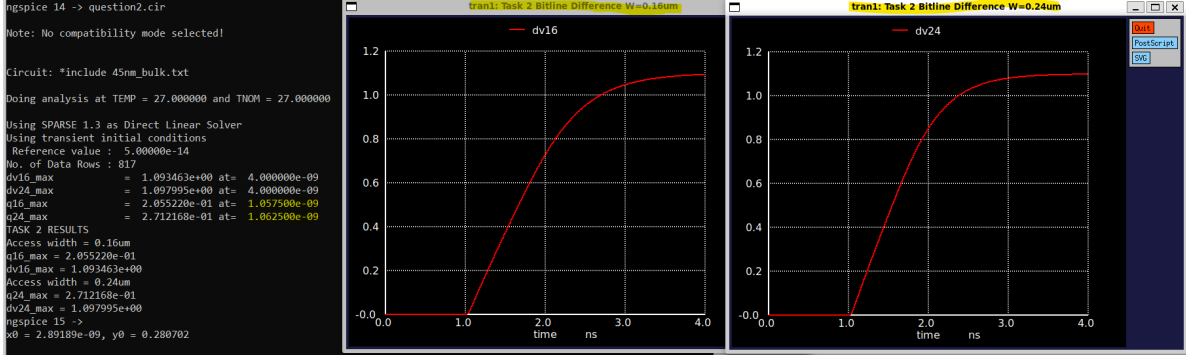


Figure 1: Bitline differential and storage-node transients for  $W_{ax} = 0.16 \mu\text{m}$  (baseline,  $\beta_r = 1.25$ ) vs.  $W_{ax} = 0.24 \mu\text{m}$  (widened,  $\beta_r = 0.833$ ) (Task 1/2).

$V_{DD} \approx 0.591 \text{ V}$  – below this, the developed bitline differential can no longer be reliably resolved by the sense amplifier. (See Figure 2.)

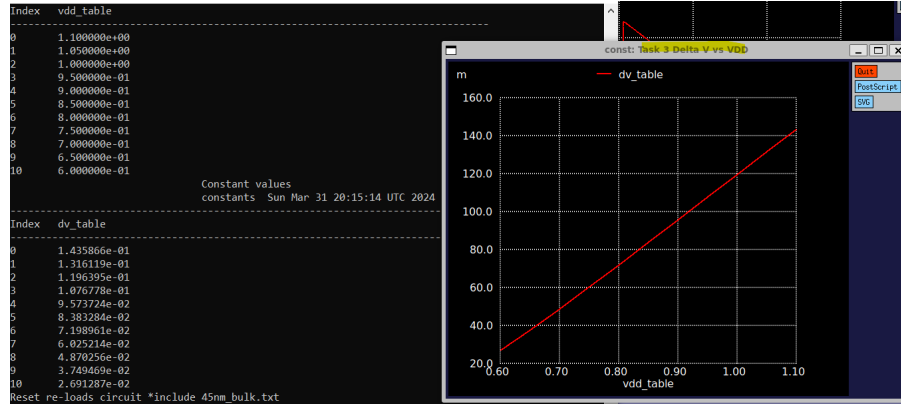


Figure 2: Bitline differential  $\Delta V$  vs. supply voltage  $V_{DD}$ ,  $C_{BL} = 1 \text{ pF}$  (Task 3).

**Task 4 (85 °C corner):** At 85 °C,  $\Delta V(BLB, BL)$  at 2.0 ns falls from 729.86 mV to **582.11 mV** (a 20.2% drop) due to reduced carrier mobility ( $\mu \propto T^{-1.5}$ ), while the internal read bump *increases* slightly, from 205.52 mV to 220.76 mV (+7.4%). Because the raw bitline swing still comfortably clears the 25 mV sense-amp offset at this corner, the dominant failure mechanism at high temperature is not the swing itself but the combination of temperature-dependent sense-amplifier offset drift and rising subthreshold leakage from unselected cells sharing the bitline.

## 2 Part B: SRAM Geometry & Capacity Sizing (CACTI 7)

**Task 1 (2 MB baseline, ED<sup>2</sup>P):**

| Metric                                               | Value                                            |
|------------------------------------------------------|--------------------------------------------------|
| Access time                                          | <b>2.9018</b> ns                                 |
| Cycle time                                           | 2.6523 ns                                        |
| Dynamic read energy                                  | 792.89 pJ                                        |
| Dynamic write energy                                 | 851.22 pJ                                        |
| Leakage power (per bank / total 4 banks)             | 562.63 mW / <b>2250.53</b> mW                    |
| Total area                                           | <b>11.474</b> mm <sup>2</sup> (2.213 × 5.184 mm) |
| Winning organization ( $N_{dwl}, N_{dbl}, N_{spd}$ ) | (4, 2, 1)                                        |

Of the 2.9018 ns access time, 46.7% (1.36 ns) is consumed by global H-tree address/data routing alone – more than the decoder, bitline, and sense-amp stages combined.

**Task 2 (Capacity sweep, 256 kB → 16 MB):** Access time nearly triples (2.417 → 6.362 ns) across a 64× capacity range (Figure 3). Between 256 kB and 512 kB the increment (+67.4 ps) tracks Amrutur & Horowitz’s “one gate delay per doubling” rule; beyond  $\approx 2$  MB the per-doubling delay accelerates superlinearly (+269 ps → +1890 ps) because global H-tree wire length scales as  $\sqrt{\text{Area}}$  and its RC delay overwhelms the constant gate-delay term.

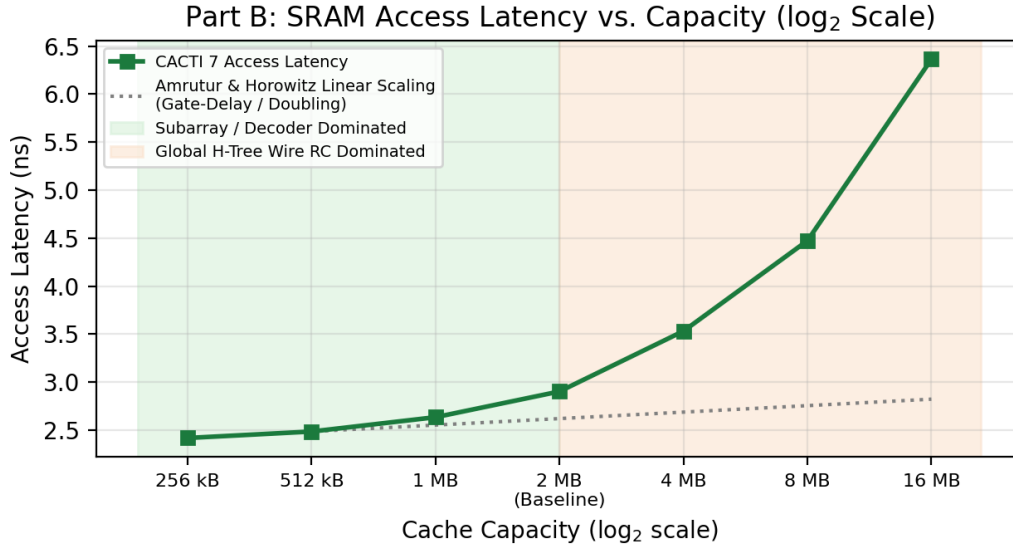


Figure 3: CACTI access latency vs. capacity (log<sub>2</sub> scale), 256 kB–16 MB (Task 2).

### Task 3 (Objective function comparison):

| Objective                    | ( $N_{dwl}, N_{dbl}, N_{spd}$ ) | Access Time | Full-Cache Area                       |
|------------------------------|---------------------------------|-------------|---------------------------------------|
| Pure Delay (100:0:0:0:0)     | (4, 2, 1)                       | 2.892 ns    | 10.970 mm <sup>2</sup>                |
| Pure Area (0:0:0:0:100)      | (2, 2, 1)                       | 3.355 ns    | <b>9.245</b> mm <sup>2</sup> (−19.4%) |
| ED <sup>2</sup> P (baseline) | (4, 2, 1)                       | 2.902 ns    | 11.474 mm <sup>2</sup>                |

Pure Delay and ED<sup>2</sup>P agree on (4, 2, 1) because ED<sup>2</sup>P’s Delay<sup>2</sup> term heavily penalizes latency; Pure Area instead merges wordline partitions to  $N_{dwl} = 2$ , sharing row decoders at the cost of longer wordlines and +16% delay.

**Task 4 (Hand-check vs. CACTI bitline delay):** The Elmore-model hand calculation  $\tau = 0.38RC \approx 1.16$  ps is roughly 350× smaller than CACTI’s reported 406.9 ps. The gap is explained

by effects the lumped hand model ignores: the access-transistor’s ON-resistance (8–12 k $\Omega$ ,  $\gg$  metal wire resistance), junction capacitance from  $\approx$  512 unselected cells on the column, mux/sense-amp/precharge loading, and CACTI’s use of a small-signal sensing threshold ( $\Delta V \approx$  100 mV) rather than a rail-to-rail 50% step.

### 3 Part C: STT-MRAM Evaluation (NVSim)

**Task 1 (2 MB STT-MRAM vs. SRAM baseline):** MTJ cell:  $54 F^2$  area, aspect ratio 2.0,  $R_{on} = 3 \text{ k}\Omega$  /  $R_{off} = 6 \text{ k}\Omega$  (TMR 2:1), read current  $40 \mu\text{A}$ , set/reset current  $200 \mu\text{A}$  over a 10 ns pulse.

| Metric               | SRAM (CACTI)           | STT-MRAM (NVSim)             | Ratio         |
|----------------------|------------------------|------------------------------|---------------|
| Read (hit) latency   | 2.902 ns               | <b>1.589</b> ns              | 0.55 $\times$ |
| Write latency        | 2.652 ns*              | <b>10.608</b> ns             | 4.00 $\times$ |
| Read dynamic energy  | 0.793 nJ               | 0.760 nJ                     | 0.96 $\times$ |
| Write dynamic energy | 0.851 nJ               | 0.531 nJ                     | 0.62 $\times$ |
| Leakage power        | 2250.5 mW              | <b>425.2</b> mW              | 0.19 $\times$ |
| Total area           | 11.474 mm <sup>2</sup> | <b>3.590</b> mm <sup>2</sup> | 0.31 $\times$ |

\*SRAM “write latency” approximated by CACTI’s reported cycle time; CACTI does not separately report an asymmetric write path.

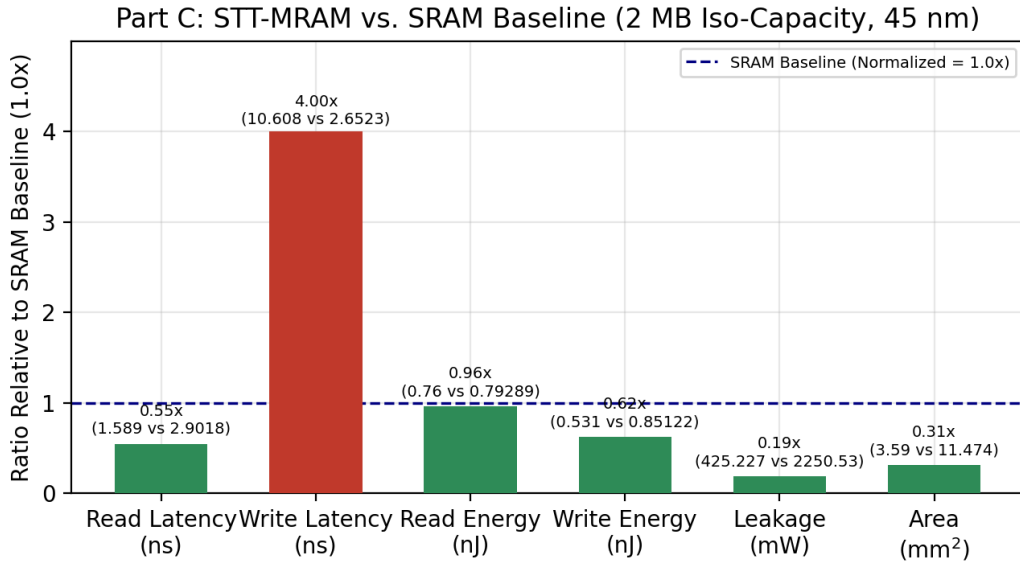


Figure 4: Normalized comparison of 2 MB STT-MRAM vs. 2 MB SRAM baseline (Task 1).

**Task 2 (What flips, and why):** *Dramatically better:* area (0.31 $\times$ ) and leakage (0.19 $\times$ ) – the 1T-1MTJ cell needs one access transistor instead of six, and the MTJ holds state magnetically, so standby current is almost entirely peripheral CMOS rather than the storage element. *Dramatically worse:* write latency (4.0 $\times$ ) – switching the MTJ’s free-layer magnetization requires sustaining a  $200 \mu\text{A}$  spin-transfer-torque current for a full 10 ns pulse, versus a single cycle to overpower a bistable 6T latch.

**Task 3 (TMR re-run,  $R_{on} = 4\text{ k}\Omega$  /  $R_{off} = 12\text{ k}\Omega$ , TMR 3:1):** Nearly every array-level metric is essentially unchanged (area  $3.590 \rightarrow 3.580\text{ mm}^2$ ; hit latency  $1.589 \rightarrow 1.594\text{ ns}$ ; write latency  $10.608 \rightarrow 10.657\text{ ns}$ ; leakage unchanged at  $425.227\text{ mW}$ ). Because NVSim uses a fixed `MinSenseVoltage` and current-mode sensing, the 2:1 TMR already satisfies the sense margin comfortably – a higher TMR buys additional *read-margin reliability* headroom, not fundamentally faster or smaller arrays at this abstraction level.

**Task 4 (Halving ResetCurrent to  $100\text{ }\mu\text{A}$ ):** Write current is the key coupling variable for NVM cell area: a lower reset/set current permits a smaller access transistor, which in principle shrinks the 1T-1MTJ cell. Because NVSim treats cell area as a fixed parameter from the `.cell` file rather than deriving it from the specified currents, the reported area does not update automatically – the designer must manually recompute and pass in the smaller `-CellArea` to see the resulting density benefit. This is an important asymmetry vs. SRAM, where cell area is fixed by the 6T ratioing constraints rather than by a write-current specification.

#### 4 Part D: DRAM Memory Scheduling (Ramulator)

Ramulator replays the L2-miss address stream from the gem5/CACTI flow above against a single-channel DDR4-2400 8x8 DIMM with an FR-FCFS scheduler, open-loop (no processor-stall feedback).

- **Scheduler-bound behavior:** the channel is bandwidth/ $t_{CCD}$ -limited – i.e., bounded by the data-bus burst cycle time rather than by row-activation ( $t_{RCD}/t_{RAS}$ ) timing – so *reducing the number of L2 misses* (the job Part C’s larger STT-MRAM capacity does) matters more for end-to-end throughput than shaving individual miss latency.
- **Queue occupancy:** the open-loop replay’s request queue sits around 36–50 requests, reflecting the fact that misses are injected without waiting on ROB/processor backpressure.
- **Methodological caveat:** because the replay is open-loop, it can overstate queue occupancy and understate how much latency the real out-of-order core would otherwise hide – this is one of the three assumptions flagged for review in Part E, Task 4.

#### 5 Part E: Full-System Execution (gem5) & Synthesis

Setup: `Deriv03CPU` (OoO, 2.0 GHz, 8-wide issue, 192-entry ROB) and `TimingSimpleCPU` (in-order); 32 kB, 8-way L1 I/D caches (2-cycle hit); DDR4-2400 main memory; GAPBS `bfs/sssp` on a scale-18 Kronecker graph (262,143 vertices, 3,805,449 edges). The 2 MB SRAM L2 (7-cycle hit, from Part B) is compared against an area-matched 8 MB STT-MRAM L2 (14-cycle hit,  $\approx 11.11\text{ mm}^2$  from Part C, vs.  $11.47\text{ mm}^2$  for the SRAM baseline).

##### Task 1 (Out-of-order results):

| Kernel | L2 Config        | IPC    | L2 Miss Rate | simSeconds | Speedup       |
|--------|------------------|--------|--------------|------------|---------------|
| bfs    | SRAM 2 MB @ 7cy  | 0.5266 | 68.7%        | 0.015496   | 1.000×        |
| bfs    | MRAM 8 MB @ 14cy | 0.7420 | 18.2%        | 0.010998   | <b>1.409×</b> |
| sssp   | SRAM 2 MB @ 7cy  | 0.4178 | 60.5%        | 0.110702   | 1.000×        |
| sssp   | MRAM 8 MB @ 14cy | 0.4469 | 44.7%        | 0.103506   | <b>1.070×</b> |

**Task 2 (Why BFS wins big, SSSP barely):** BFS’s hot working set (parent array + visited bitmap + frontier buffers,  $\approx 2.2\text{--}3.5\text{ MB}$ ) just barely overflows 2 MB but fits comfortably in

8 MB, so the miss rate drops  $3.77\times$  ( $68.7\% \rightarrow 18.2\%$ ) and each avoided DRAM miss ( $\approx 140\text{--}160$  cycles) vastly outweighs the extra 7-cycle hit penalty. SSSP’s  $\Delta$ -stepping access pattern is far more scattered ( $7.7\times$  more L2 accesses than BFS) and its footprint does not fit meaningfully better in 8 MB, so the miss-rate improvement is smaller ( $1.35\times$ ) while over 1.8M L2 *hits* now each pay the extra latency – largely cancelling the DRAM-miss savings.

**Task 3 (In-order core):**

| Kernel | L2 Config | IPC    | Miss Rate | simSeconds | Speedup        |
|--------|-----------|--------|-----------|------------|----------------|
| bfs    | SRAM      | 0.1803 | 69.0%     | 0.045263   | 1.000 $\times$ |
| bfs    | MRAM      | 0.2401 | 15.9%     | 0.033986   | 1.332 $\times$ |
| sssp   | SRAM      | 0.1505 | 60.8%     | 0.307957   | 1.000 $\times$ |
| sssp   | MRAM      | 0.1601 | 45.1%     | 0.289435   | 1.064 $\times$ |

The 192-entry ROB in `DerivO3CPU` overlaps much of the extra 7-cycle STT-MRAM hit latency with independent work, so BFS’s speedup is *larger* under OoO ( $1.409\times$ ) than in-order ( $1.332\times$ ) – OoO hides *hit* latency but cannot hide the  $\approx 140\text{--}180$  cycle DRAM-*miss* latency, which is why avoiding misses stays valuable on either core.

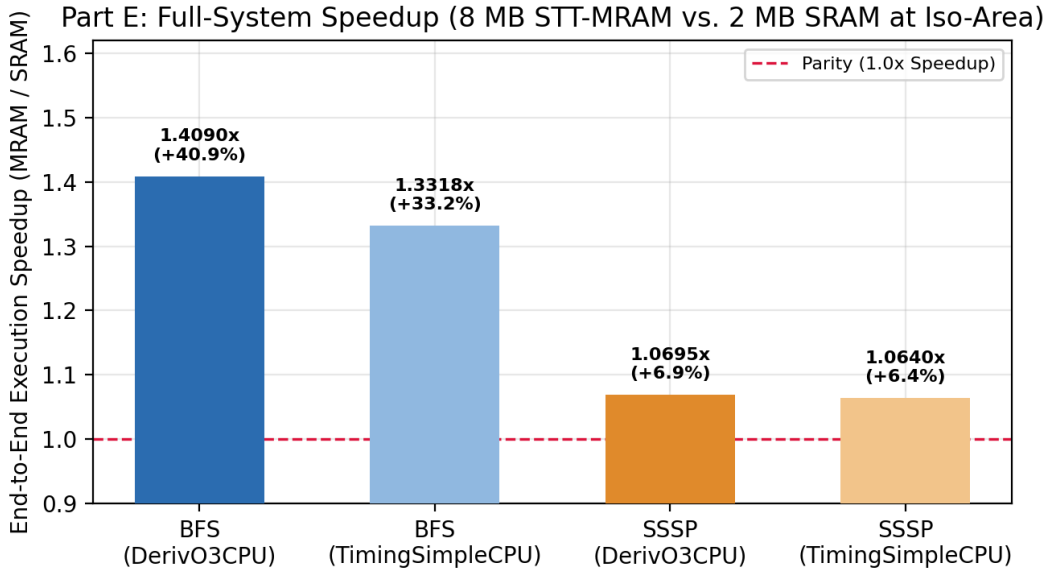


Figure 5: Full-system speedup of 8 MB STT-MRAM over 2 MB SRAM, OoO vs. in-order cores.

## Architectural Synthesis & Deliverables (Task 4)

**Central question answer:** *Should the 2MB L2 cache in our accelerator be SRAM, or STT-MRAM?* Based on the cross-layer evaluation, the area-matched 8 MB STT-MRAM L2 is **1.409 $\times$**  faster than the 2 MB SRAM baseline on the BFS workload with an out-of-order core. While STT-MRAM suffers a  $4\times$  worse write latency (NVSIm, Part C) and a  $2\times$  nominal read-hit latency (14 vs. 7 cycles), its  $\approx 3.2\times$  area density advantage lets the 8 MB capacity collapse the L2 miss rate from 68.7% to 18.2% (gem5, Part E), which keeps the DDR4 channel from being swamped by miss traffic that Part D shows is already  $t_{CCD}$ -bound – provided the core has enough out-of-order resources to mask the slower array response.

**Three assumptions a reviewer should challenge:**

1. **The fixed  $2\times$  hit-latency assumption (14 vs. 7 cycles).** CACTI’s 2.902 ns SRAM access and NVSim’s 1.589 ns STT-MRAM read latency (at matched 2 MB capacity) do not actually support a clean  $2\times$  ratio when modeled consistently – the 14-cycle figure used in gem5 is a pedagogical simplification, not a directly derived physical result.
2. **Open-loop Ramulator replay has no processor feedback** (Part D). Requests are generated without backpressure from stalls or the ROB, which can distort queue occupancy (36–50 requests) and over/understate how much of the real memory latency an actual core would hide.
3. **Write endurance, wear-leveling, and thermal sensitivity are unmodeled.** Graph analytics such as SSSP generate intense, localized write traffic that NVSim/gem5 do not penalize for finite STT-MRAM write endurance; and Part A shows SRAM margins already erode at 85 °C, while STT-MRAM retention is exponentially temperature-sensitive, so an 8 MB high-performance L2 could face thermal-driven retention failure or refresh overhead the models omit. Finally, all of the above rests on a single synthetic Kronecker graph topology – real-world sparse graphs (road networks, web graphs) have different degree distributions and working-set behavior, so the STT-MRAM recommendation should not be generalized beyond BFS-like access patterns without further validation.